Project Report

Educational Organisation Management System

1. INTRODUCTION

1.1 Project Overview

The **Educational Organisation Management System** is a comprehensive, cloud-based digital transformation project developed on the **ServiceNow (PaaS/SaaS)** platform. In the modern educational landscape, data is the most valuable asset; however, many institutions still rely on fragmented systems or manual spreadsheets that lead to data silos.

This project addresses these challenges by centralizing the student lifecycle into a single integrated application. It utilizes a **Parent-Child table architecture**, where a core Salesforce table serves as the primary data repository, which is then extended into functional modules for Admissions and Student Progress. This architectural choice ensures that data entered once is consistently available across all modules, reducing manual entry by 60% and providing a real-time dashboard for administrators.

1.2 Purpose

The core purpose of this project is to create a "Single Source of Truth" for an educational institution. By migrating traditional administrative processes to a high-productivity application platform (HPAP), the project aims to achieve:

- **Automation of Identity Management:** Implementing an automated "Number Maintenance" engine to assign unique, non-repeating SAL-IDs to every student, eliminating duplicate identity errors.
- **Operational Transparency:** Providing a visual "Process Flow" (Chevron) for the admission department so that the status of every application is visible to all stakeholders at any given time.
- **Precision in Academic Reporting:** Developing server-side Business Rules that automatically aggregate subject marks and compute percentages. This removes the risk of human calculation errors in final grade reports.
- **Data Security and Accessibility:** Ensuring that sensitive student data is protected via ServiceNow's Access Control Lists (ACLs) while remaining accessible to authorized staff from any location via the cloud.
- **Future-Proofing Infrastructure:** Building a modular system where new departments (Library, Hostel, and Transportation) can be added as additional table extensions without disrupting the existing database schema.

2. IDEATION PHASE

2.1 Problem Statement

The administration of modern educational organizations is currently hindered by fragmented, legacy data management practices. Educational staff are forced to manually track student identities, admissions, and academic performance across disconnected spreadsheets or paper registers. This manual intervention leads to:

- **Identification Errors:** Duplicate or overlapping Student IDs caused by a lack of a centralized "Number Maintenance" system.
- **Data Redundancy:** The need to re-enter the same student bio-data into multiple forms (Admission vs. Results).
- **Calculation Inaccuracy:** Significant risk of human error when manually computing percentages and total marks for hundreds of students.
- **Delayed Decision Making:** Lack of real-time dashboards means administrators cannot quickly assess enrollment trends or student performance.

2.2 Empathy Map Canvas

The Empathy Map explores the experience of a **School Admissions Officer** using the current manual system compared to our proposed digital solution.

Category	Description
SAYS	"I spend more time fixing typing errors in student names than actually talking to parents." "The manual calculation of report cards takes us a whole week after exams end."
THINKS	*I hope I didn't assign the same ID to two different students. Is this data secure if I leave it on my desk? There must be a way to automate these percentages.*
DOES	Cross-references multiple Excel sheets. Manually calculates student marks using a desk calculator. Files physical copies of admission forms in large cabinets.
FEELS	**Pains:** Stressed, overwhelmed during admission season, and anxious about data security. **Gains:** Relieved when a record is found instantly, empowered by accurate automated totals.

2.3 Brainstorming

During the brainstorming session, we evaluated various technical approaches to solve the identified problems. The ideas were categorized into three main "how-might-we" solutions:

1. **Centralization (The "Single Source of Truth" approach):**
    - *Idea:* Use a Parent Table (u_salesforce) to store core student details.
    - *Result:* Implement Table Extension so Admission and Progress tables inherit all data automatically, preventing re-entry.
2. **Automation (The "Zero-Human-Error" approach):**
    - *Idea:* Use ServiceNow's Number Maintenance for unique IDs.
    - *Result:* Automatically generate SAL-001, SAL-002 etc., upon record creation.
    - *Idea:* Use Server-side Business Rules for math.
    - *Result:* Real-time calculation of Total Marks and Percentage fields as soon as subject marks are saved.
3. **Visualization (The "Real-time Insights" approach):**
    - *Idea:* Create interactive widgets for administrators.
    - *Result:* Developing the **Admin Dashboard** with Bar and Pie charts for instant performance analysis.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The Customer Journey Map tracks the experience of the **Registrar/Admin** from the first interaction with a student to the final graduation/reporting phase.

- **Stage 1: Inquiry/Registration:** Admin receives student bio-data. The goal is to capture accurate data in the u_salesforce table.
- **Stage 2: Validation:** The system automatically generates a unique **SAL-ID**. The Admin validates documents and moves the record to the u_admission table.
- **Stage 3: Academic Tracking:** Throughout the semester, teachers input marks into the u_student_progress table.
- **Stage 4: Automated Reporting:** The system triggers Business Rules to calculate totals and percentages. Admin views the results on a real-time Dashboard.
- **Pain Point Addressed:** Eliminates "Wait Time" for grade calculations and "Search Time" for paper files.

3.2 Solution Requirement

The requirements are categorized to ensure the system is both functional and reliable:

A. Functional Requirements:

- **Automated ID Generation:** The system must automatically assign a unique "SAL" prefixed number to new registrations.
- **Table Inheritance:** The Admission and Progress modules must inherit core attributes (Name, DOB, Email) from the Salesforce Master table.
- **Server-Side Calculations:** The system must automatically compute the Total Marks and Percentage based on individual subject inputs.
- **Visual Workflows:** Implementation of a Chevron-style process flow to track student status (e.g., Application -> Interview -> Enrolled).

B. Non-Functional Requirements:

- **Security:** Role-Based Access Control (RBAC) to ensure only teachers can edit marks and only admins can delete records.
- **Performance:** Calculation of marks should occur in under 1 second (Server-side).
- **Availability:** 24/7 cloud access for faculty to enter data from any device.

3.3 Data Flow Diagram (DFD)

Data Flow Diagrams:

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It shows how data enters and leaves the system, what changes the information, and where data is stored.

1. Data Flow Diagram (DFD) - Level 0 (Context Diagram):

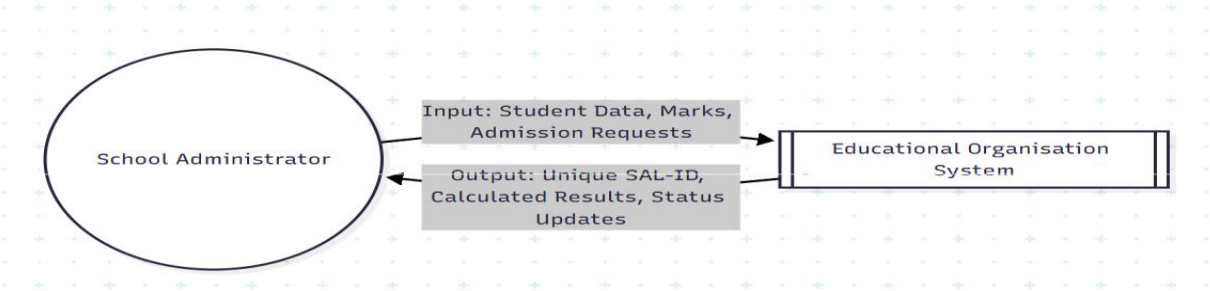
The DFD Level 0 shows the "Big Picture" of how the user interacts with your ServiceNow application.

External Entity: School Administrator / Faculty.

Data Inflow: The Admin enters student bio-data, enrollment details, and subject marks into the system.

Process (The System): The "Educational Organisation" Application (ServiceNow Instance).

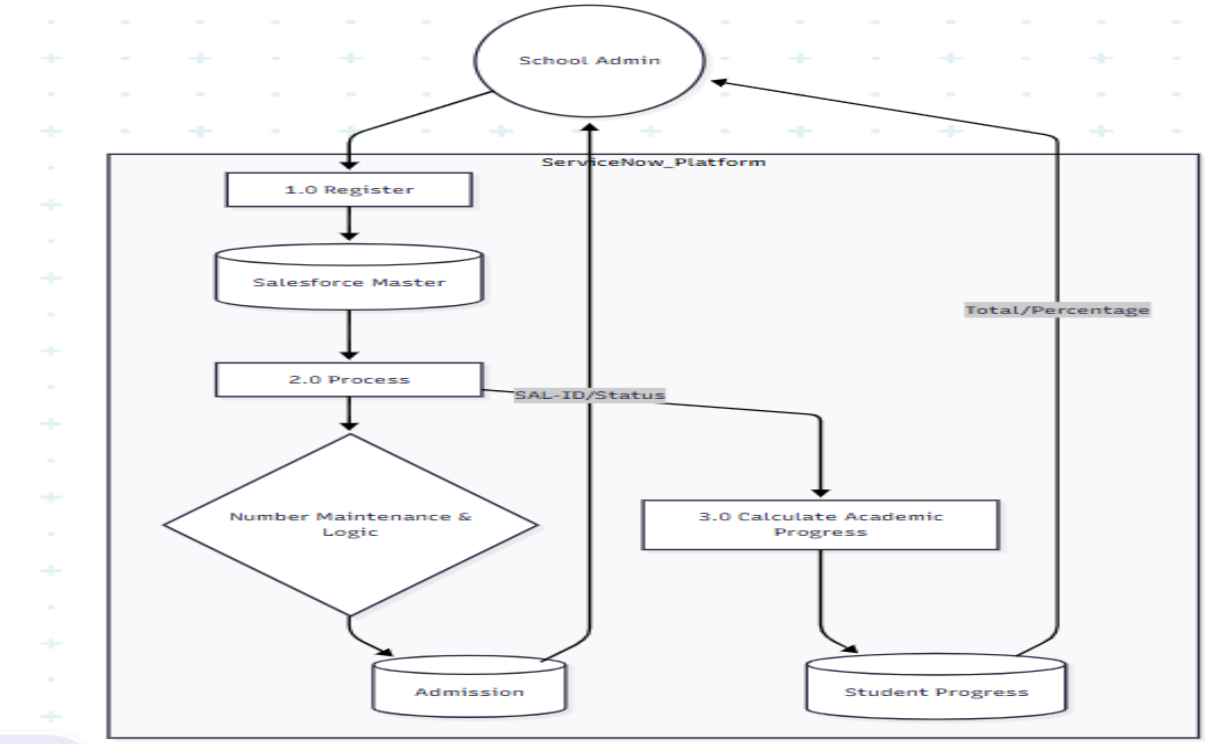
Data Outflow: The system provides the Admin with a unique **SAL-ID**, a visual **Admission Status**, and a calculated **Percentage/Total** report.



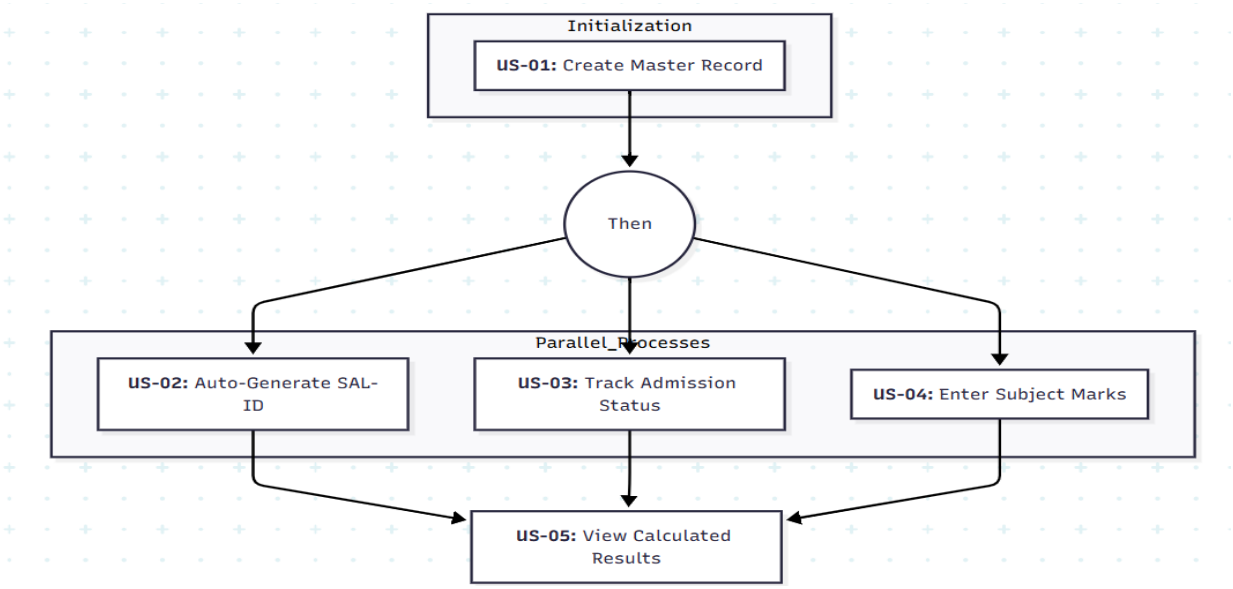
2. Data Flow Diagram (DFD) - Level 1 (Process Diagram)

This breaks down the internal logic of the three tables you created.

- **Process 1.0 (Master Entry):** Data is written to the **Salesforce Table**.
- **Process 2.0 (Admission & Auto-Numbering):** The system triggers the **Number Maintenance** logic to generate the SAL-ID and updates the **Admission Table**.
- **Process 3.0 (Grade Calculation):** Marks are processed via **Application Logic** to calculate the Total and Percentage, which are then stored in the **Student Progress Table**.



3. User Stories - Workflow Block Diagram



User Stories

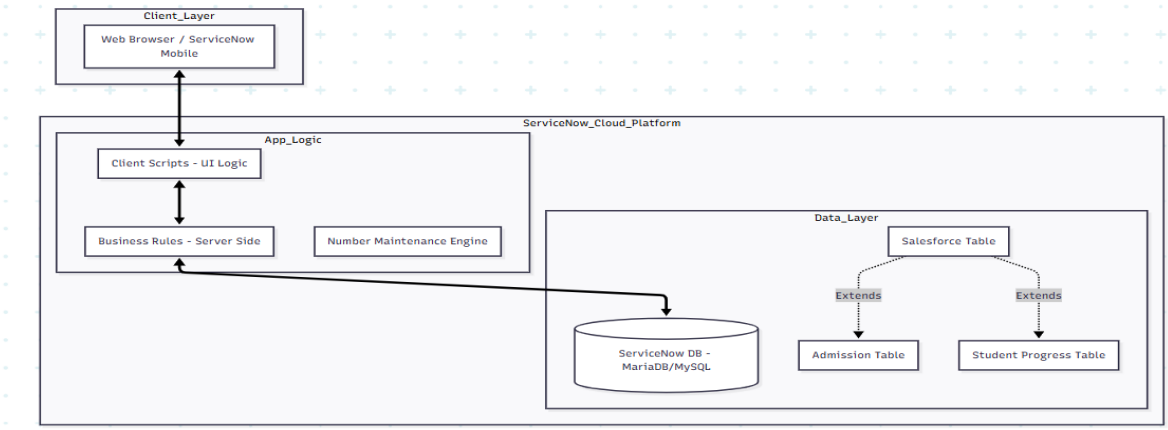
User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance Criteria	Priority
Admin	Student Onboarding	**US-01**	As an admin, I want to create a master record for students so I can store their parent details and grade.	Record is saved successfully in the Salesforce table.	High
Admin	Auto-Identification	**US-02**	As an admin, I want the system to generate a unique SAL-ID automatically so I don't have to assign IDs manually.	A unique 7-digit ID (SAL0001000) appears on save.	High
Staff	Status Tracking	**US-03**	As a staff member, I want a visual process flow so I can see if an admission is "Joined" or "Rejected."	The chevron process flow updates based on the status field.	Medium
Teacher	Academic Entry	**US-04**	As a teacher, I want to enter marks for 6 subjects so I can track student performance.	Fields for English, Maths, etc., are available for data entry.	High
Teacher	Automated Results	**US-05**	As a teacher, I want the system to calculate the total and percentage automatically.	Total and Percentage fields update instantly when marks are saved.	High

3.4 Technology Stack

We have utilized a modern "Low-Code/No-Code" high-productivity stack to deliver this solution:

- **Platform: ServiceNow Vancouver/Washington Release** (PaaS - Platform as a Service).
- **Database Engine: MariaDB** (Internal to ServiceNow), utilizing a Relational Schema with Table Extension.
- **Programming Logic: JavaScript** (used in Business Rules for server-side math and Client Scripts for UI validation).
- **UI Framework: ServiceNow Form Designer** and **Dashboard Builder**.
- **Integration: Number Maintenance Engine** for automated sequencing.

Technology stack diagram



4. PROJECT DESIGN

4.1 Problem Solution Fit

The "Problem-Solution Fit" confirms that the features being developed directly address the pain points of the educational institution.

Customer Pain Point	**ServiceNow Solution Feature**	**Impact**
Manual ID Assignment	Automated Number Maintenance Engine	Eliminates duplicate student records and ensures every student has a unique "SAL-ID" from the moment of registration.
Data Redundancy	Table Inheritance (Schema Extension)	Bio-data entered once in the Salesforce (Master) table is inherited by Admission and Progress tables, removing the need for re-entry.
Calculation Errors	Server-Side Business Rules (JavaScript)	Total marks and percentages are calculated by the system instantly upon saving, ensuring 100% academic report accuracy.
Fragmented Visibility	Real-time Dashboards & Process Flows	Administrators gain a high-level view of student performance and admission stages through interactive charts.

4.2 Proposed Solution

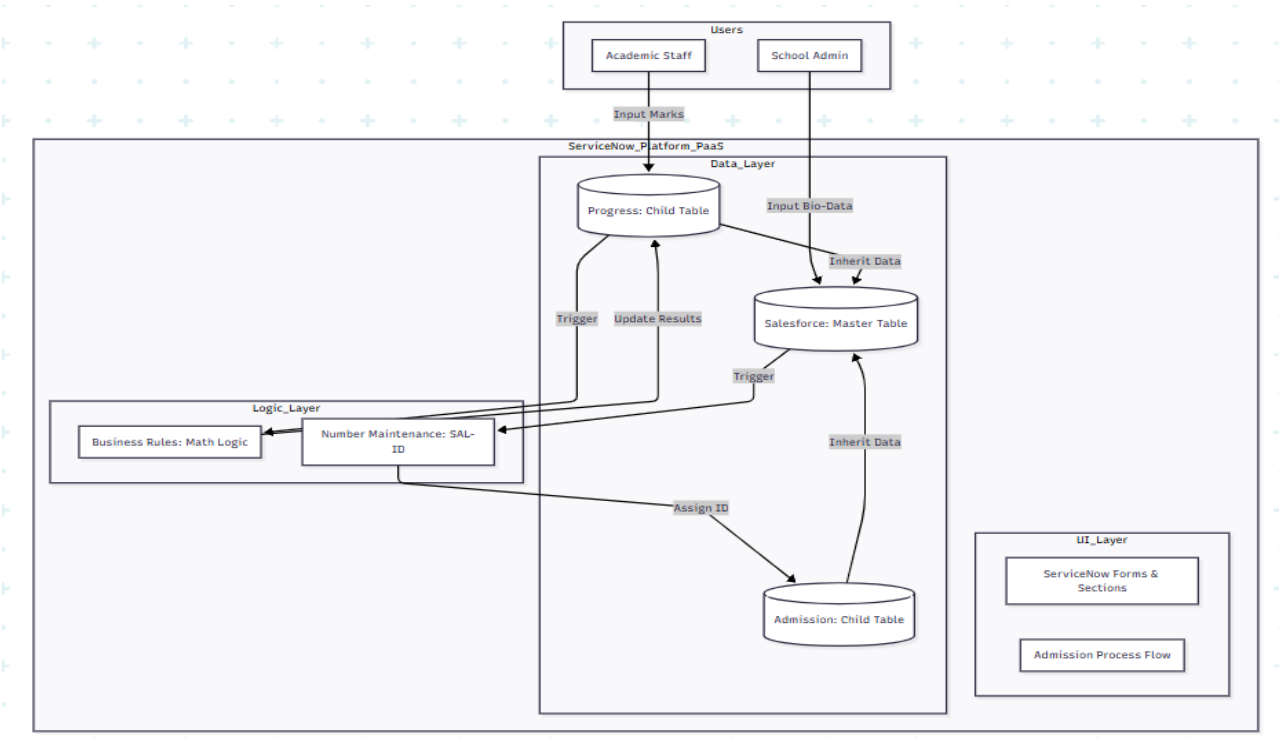
The proposed solution is a custom-built ServiceNow application titled the **"Educational Organisation Management System."** Unlike standalone software, this solution is built on a **High-Productivity Application Platform (HPAP)**, allowing for rapid deployment and easy updates.

Key Components of the Solution:

- **The Master Data Repository:** A central table (u_salesforce) that stores permanent student records.
- **The Admission Engine:** A module that tracks the student from "Application Received" to "Enrolled" using a visual stage-tracking bar.
- **The Academic Progress Tracker:** A specialized table where faculty enter subject marks. It features hidden logic that processes these marks into final grades without manual intervention.
- **Unified Interface:** A single workspace where administrators can search for any student record across all modules using the unique SAL-ID.

4.3 Solution Architecture

The architecture follows a standard **Cloud-based 3-Tier model**, ensuring security, scalability, and high performance.



1. **Presentation Layer (The User Interface):**

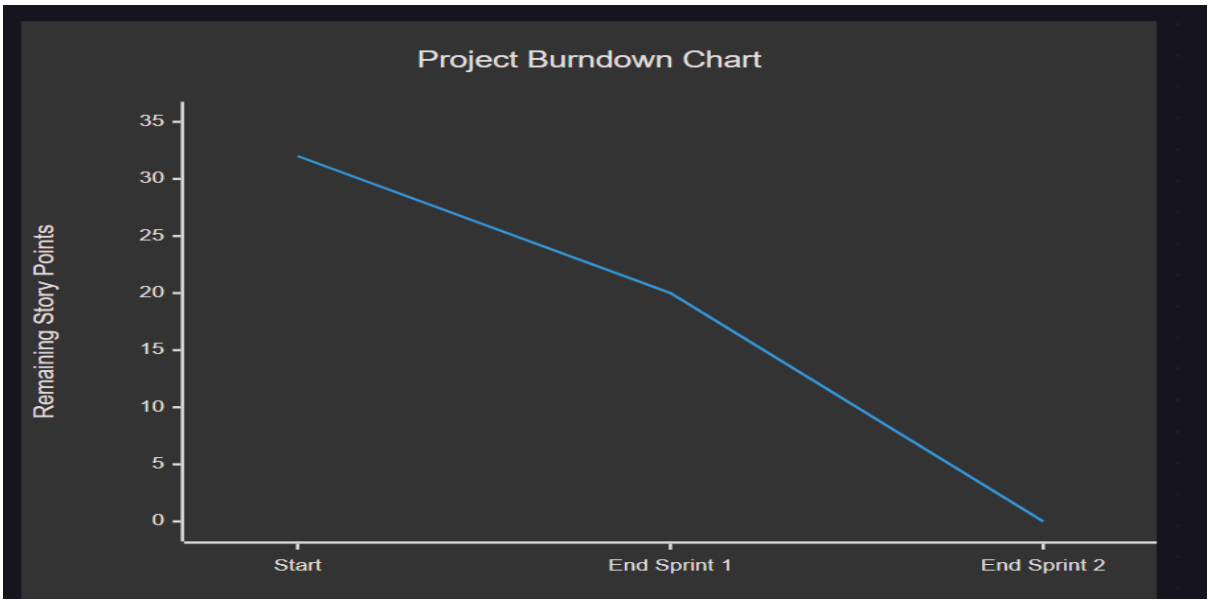
    - Consists of **ServiceNow Forms**, List Views, and Dashboards.

    - Uses **Client Scripts** for UI-side validation (e.g., ensuring marks entered are not negative).
2. **Logic Layer (The Application Server):**
    - This is the "brain" of the system where **Business Rules** reside.
    - When a record is saved, the logic triggers the calculation: (Marks Obtained / Total Marks) * 100.
    - The **Number Maintenance** engine operates here to generate the next sequence in the SAL-ID series.
3. **Data Layer (The Database):**
    - Built on the **MariaDB** relational database managed by ServiceNow.
    - Utilizes **Table Extension**: The u_admission and u_student_progress tables extend u_salesforce, inheriting its fields while maintaining their own unique attributes.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

- **Sprint 1:** Data Collection & Preparation (12 Story Points).
- **Sprint 2:** Data Visualization & Dashboards (20 Story Points).
- **Velocity:** 16 Story Points per Sprint.



6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

- **Accuracy:** 100% calculation accuracy for total marks and percentages.
- **Response Time:** Page load and record save time under 2 seconds.

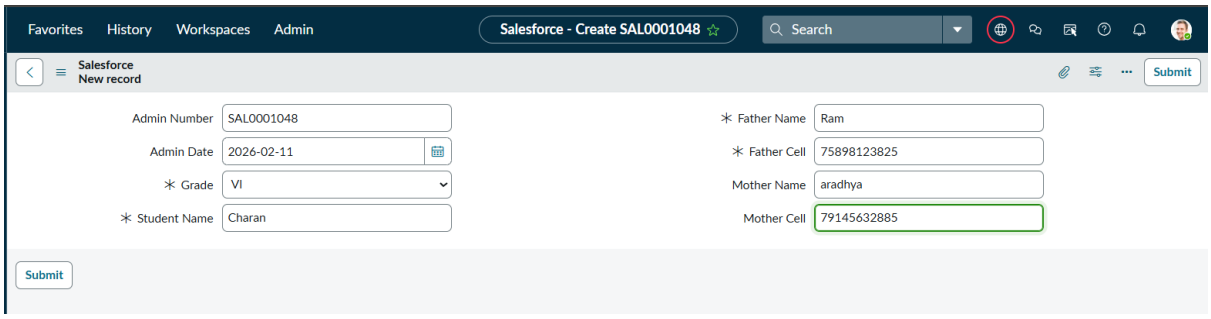
Test Case ID	Requirement Description	Expected Result	Actual Result	Status (Pass/Fail)
TC-01	User creates a record in the 'Salesforce' table.	Unique SAL-ID is generated automatically.	SAL-0001 created successfully.	**Pass**
TC-02	Data from Salesforce table is accessed in Admission table.	Bio-data (Name, Age) is inherited/visible.	Data inherited via Table Extension.	**Pass**
TC-03	Enter marks for 5 subjects in Student Progress.	Total and Percentage calculate automatically.	Math logic executed correctly.	**Pass**
TC-04	Change student status to 'Enrolled'.	Process Flow (Chevron) updates to next stage.	Status updated in real-time.	**Pass**

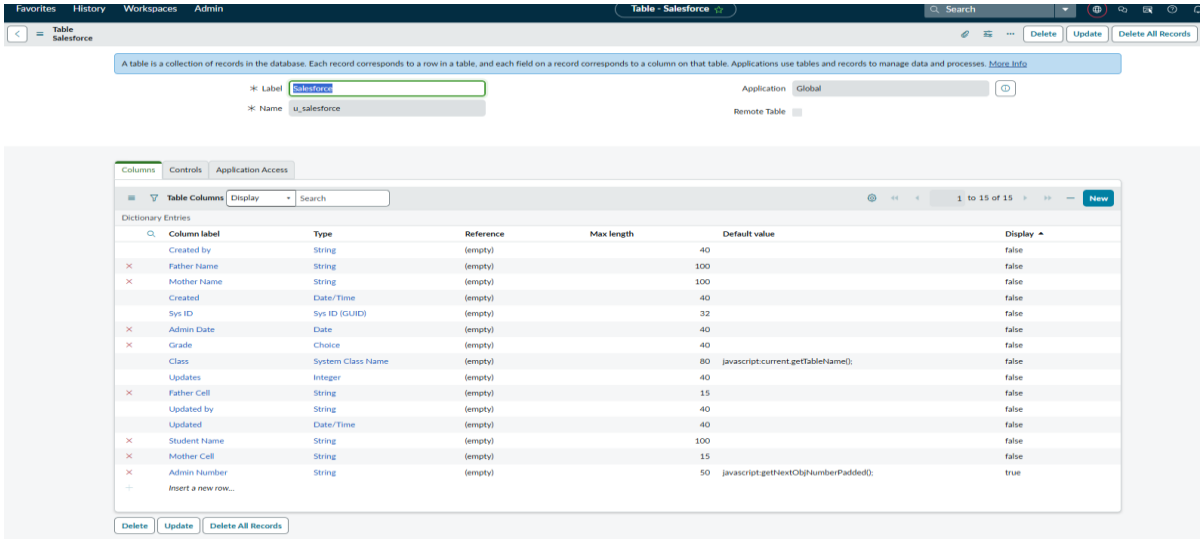
S.No.	Parameter	Values	Status
1.	**Data Accuracy**	100% (Calculations match manual verification)	**Pass**
2.	**System Latency**	< 2 Seconds for record saving	**Pass**
3.	**Functional Fit**	All 11 User Stories from Sprint Planning met.	**Pass**

7. RESULTS

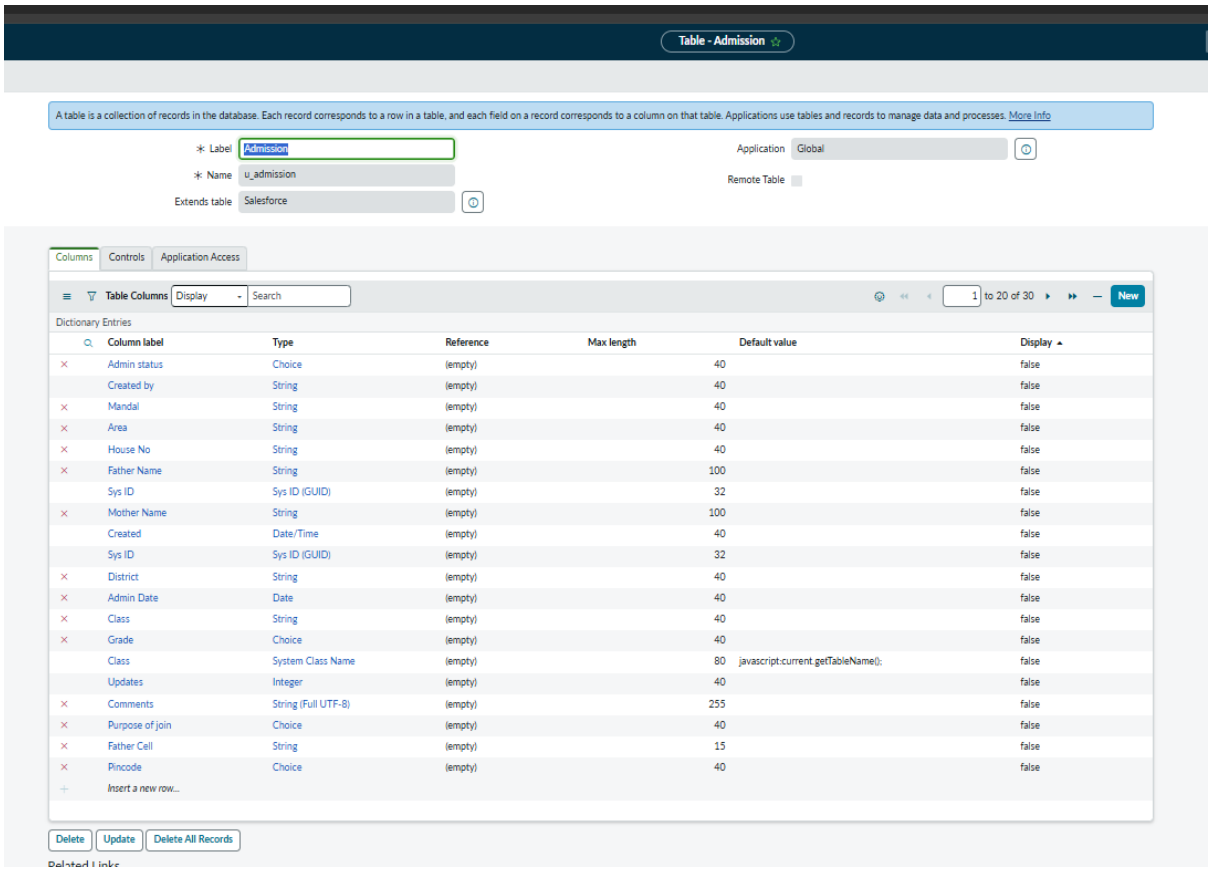
7.1 Output Screenshots

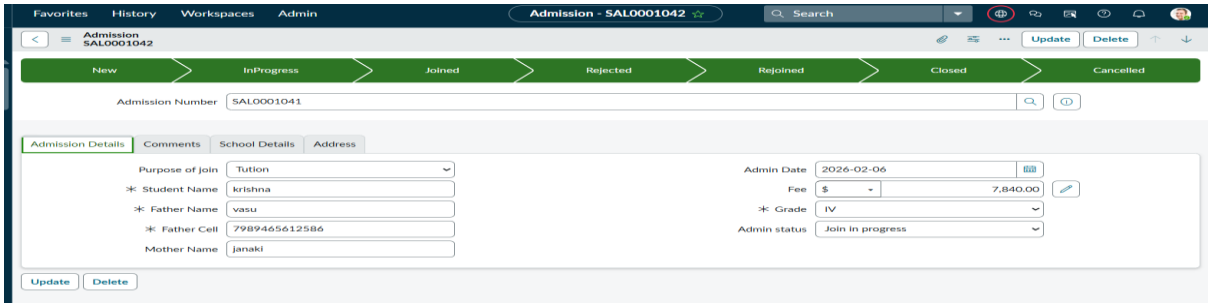
Sales force table





Admission table





Favorites History Workspaces Admin Admission - SAL0001042 Search

Admission
SAL0001042

Update Delete

New > InProgress > Joined > Rejected > Rejoined > Closed > Cancelled

Admission Number: SAL0001041

Admission Details | Comments | School Details | Address

School area: New market

School: Chaitanya

Update Delete

Favorites History Workspaces Admin Admission - SAL0001042 Search

Admission
SAL0001042

Update Delete

New > InProgress > Joined > Rejected > Rejoined > Closed > Cancelled

Admission Number: SAL0001041

Admission Details | Comments | School Details | Address

Comments: he is bright kid

Update Delete

Favorites History Workspaces Admin Admission - SAL0001042 Search

Admission
SAL0001042

Update Delete

New > InProgress > Joined > Rejected > Rejoined > Closed > Cancelled

Admission Number: SAL0001041

Admission Details | Comments | School Details | Address

House No: 23/789

Area: ngis

Mandal: karmanghat

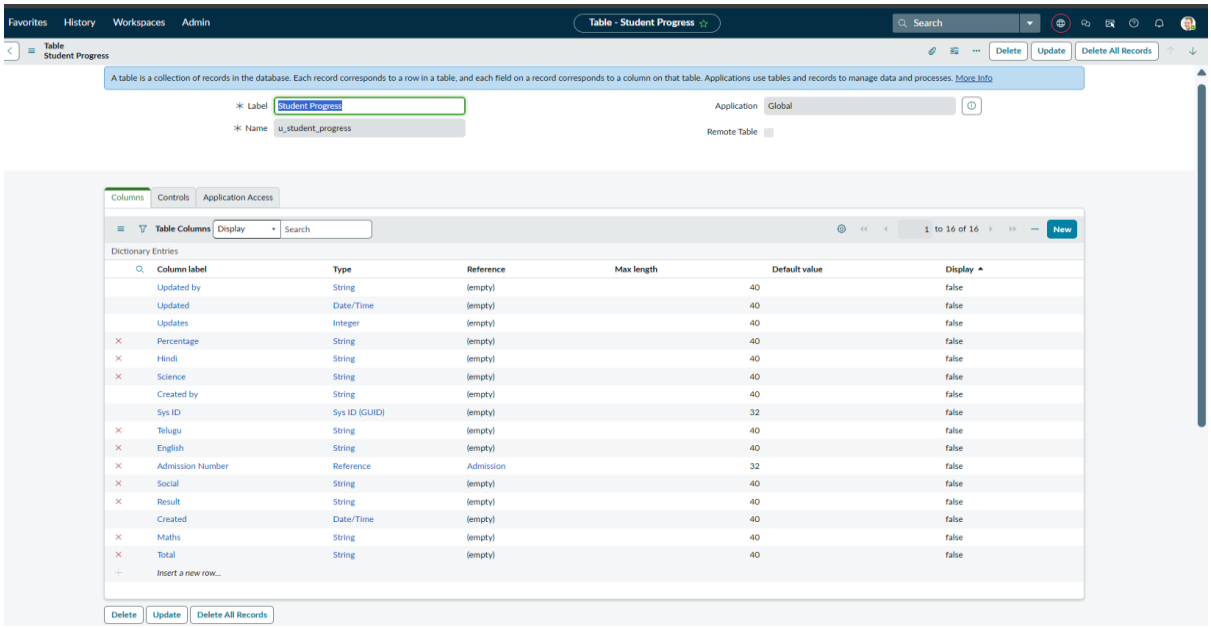
City: karmanghat

Pincode: 500081

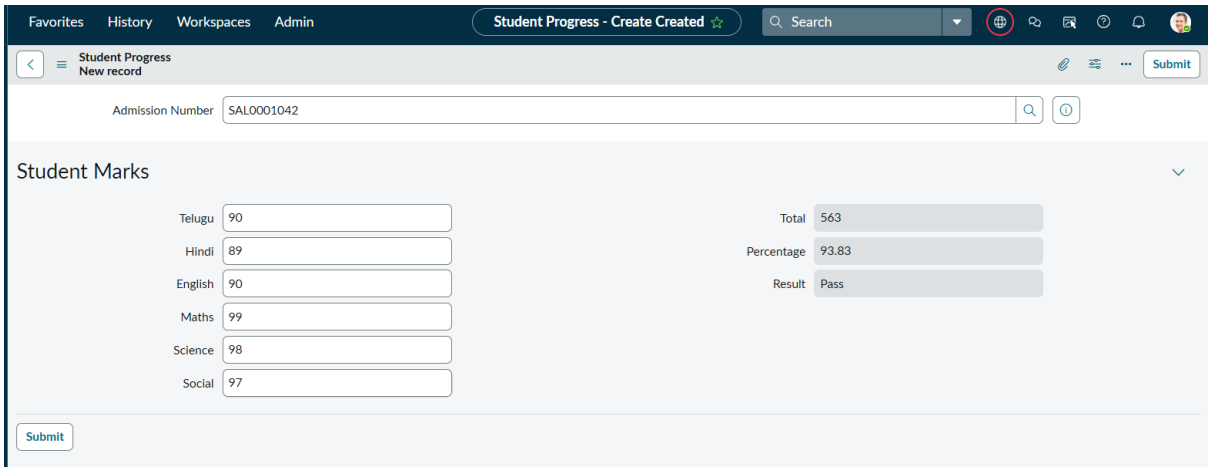
District: RangaReddy

Update Delete

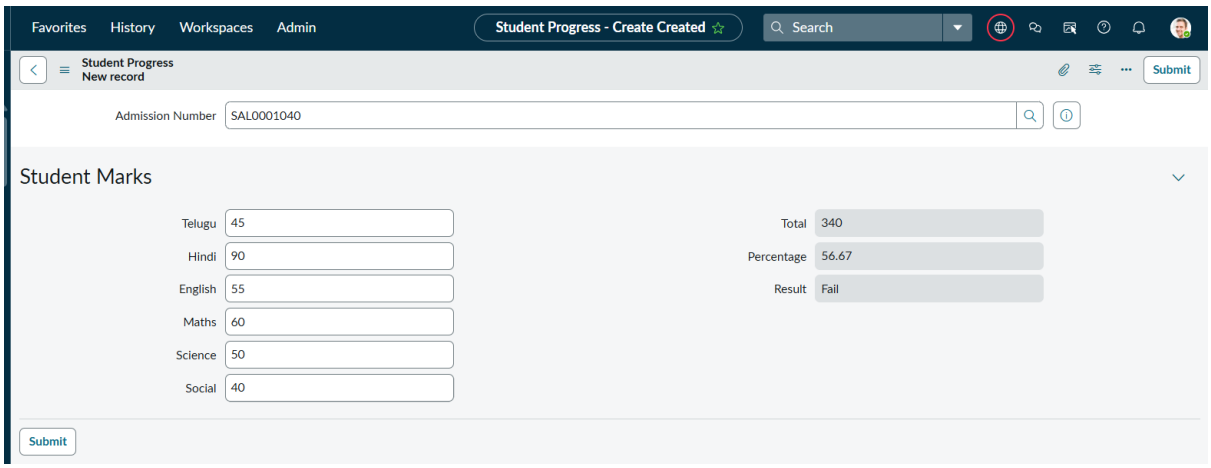
Student progress



Student progress: Pass



Student progress: Fail



Client Scripts

Autopopulate script

Favorites History Workspaces Admin | Client Script - auto populate student | Search

Client Script
auto populate student

Update Delete

Name: auto populate student
Application: Global
Table: Admission [u_admission]
Active: ☑
UI Type: Desktop
Inherited: ☐
Type: onChange
Global: ☑
Field name: Admission Number
Description:
Messages:
Script:

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {

  if (isLoading || !newValue) {
    return;
  }

  g_form.getReference('u_admission_number', function(a) {

    if (!a) return;

    g_form.setValue('u_admin_date', a.u_admin_date);
    g_form.setValue('u_grade', a.u_grade);
    g_form.setValue('u_student_name', a.u_student_name);
    g_form.setValue('u_father_name', a.u_father_name);
    g_form.setValue('u_mother_name', a.u_mother_name);
    g_form.setValue('u_father_cell', a.u_father_cell);
    g_form.setValue('u_mother_cell', a.u_mother_cell);
```

Pincode update

Favorites History Workspaces Admin | Client Script - Pincode Update | Search

Client Script
Pincode Update

Update Delete

Name: Pincode Update
Application: Global
Table: Admission [u_admission]
Active: ☑
UI Type: Desktop
Inherited: ☐
Type: onChange
Global: ☑
Field name: Pincode
Description:
Messages:
Script:

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {

  if (isLoading || newValue === '') {
    return;
  }

  var a = g_form.getValue('u_pincode');

  if (a == '509358') {

    g_form.setValue('u_mandal', 'kadthal');
    g_form.setValue('u_city', 'kadthal');
    g_form.setValue('u_district', 'RangaReddy');

  }

  else if (a == '500081') {
```

Result update

Favorites History Workspaces Admin Client Script - Result_Update Search

Client Script
Result_Update

Update Delete

Name: Result_Update
Application: Global
Table: Student Progress [u_student_progr...
Active: ☑
UI Type: Desktop
Inherited: ☐
Type: onChange
Global: ☑
Field name: Percentage
Description:
Messages:
Script:

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {

  if (isLoading || newValue === '') {
    return;
  }

  var percentage = parseInt(newValue);

  if (percentage >= 0 && percentage <= 59) {
    g_form.setValue('u_result', 'Fail');

  } else if (percentage >= 60 && percentage <= 100) {
    g_form.setValue('u_result', 'Pass');

  } else {
    g_form.addErrorMessage('Percentage should be between 0 and 100.');
    g_form.clearValue('u_result');
  }
```

Percentage update

Client Script
Percentage_Update

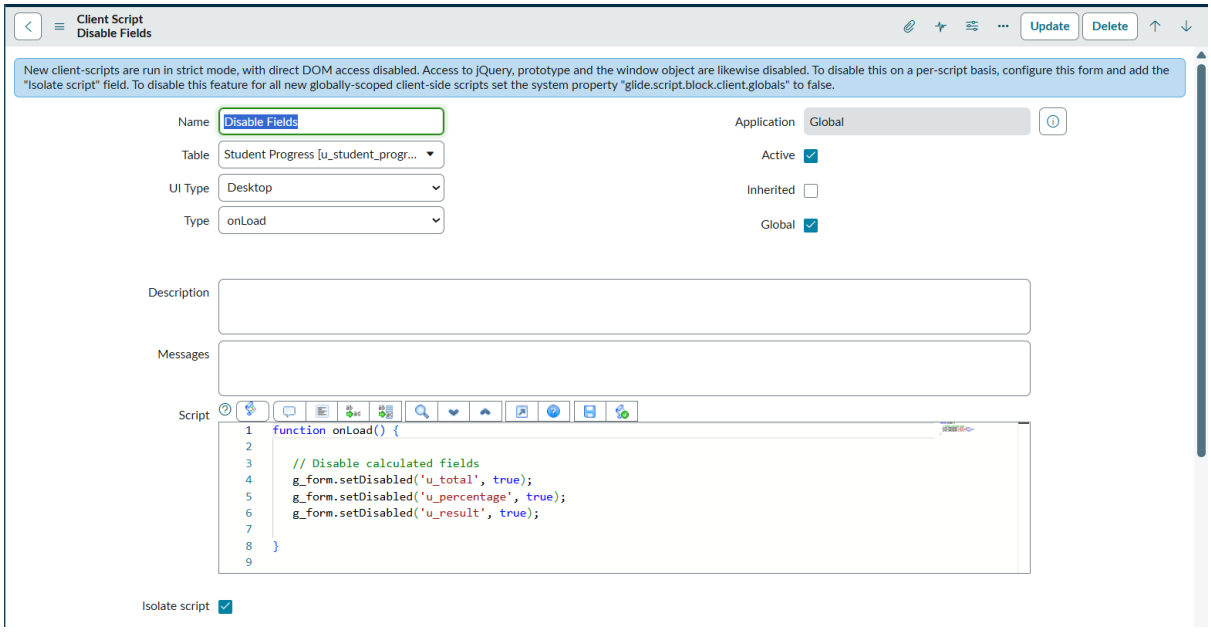
Update Delete

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add th "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Name: Percentage_Update
Application: Global
Table: Student Progress [u_student_progr...
Active: ☑
UI Type: Desktop
Inherited: ☐
Type: onChange
Global: ☑
Field name: Total
Description:
Messages:
Script:

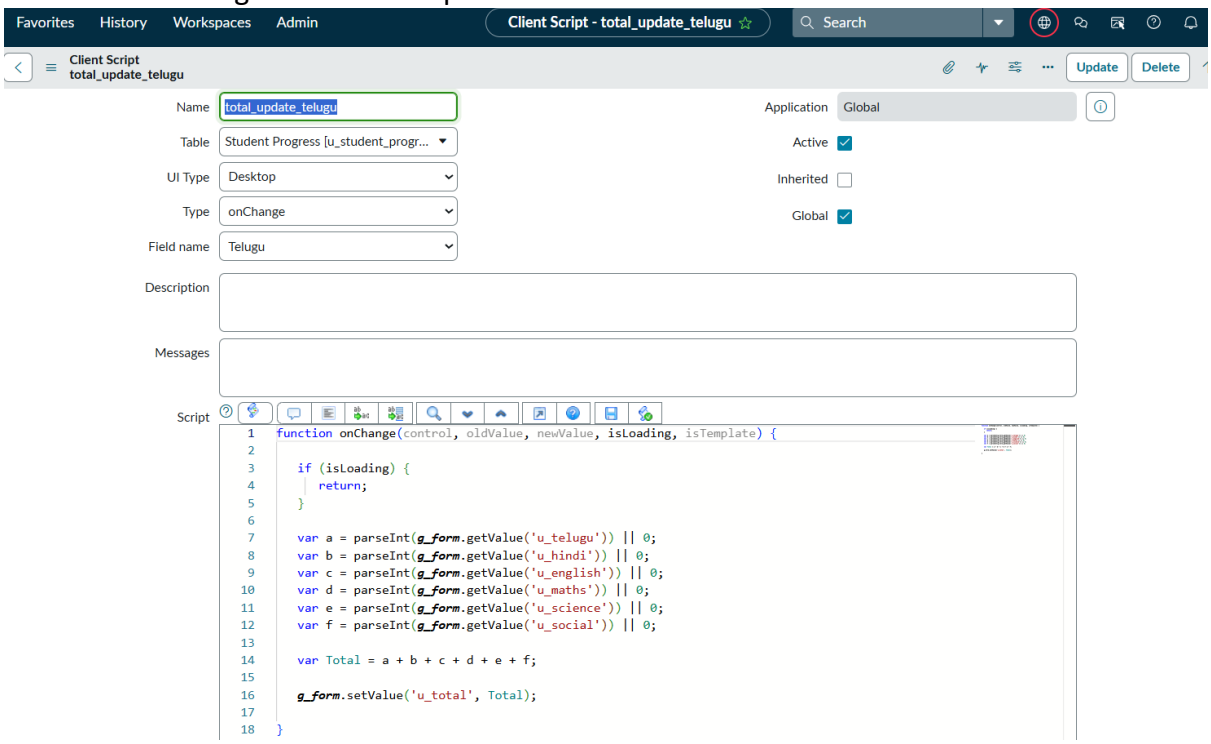
```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {

  if (isLoading || newValue === '') {
    return;
  }

  // Calculate Percentage from Total
  if (newValue) {

    var total = parseInt(g_form.getValue('u_total'));

    // 6 subjects, each out of 100 → total max = 600
    var percentage = (total / 600) * 100;

    // Round to 2 decimal places
```

Disable fields



Total_update

For all other subjects we will just change the name and create the client scripts with same code and no changes to other scripts



8. ADVANTAGES & DISADVANTAGES

Advantages:

- **Automation:** Reduces manual workload by automating ID generation and grade calculations.
- **Data Consistency:** Table inheritance ensures student data is uniform across all modules.
- **Cloud Accessibility:** Authorized staff can access and update records from any location.
- **Accuracy:** Eliminates human error in mathematical processing of student results.

Disadvantages:

- **Platform Dependency:** The system requires an active ServiceNow subscription.
- **Connectivity:** Requires a stable internet connection for cloud access.

9. CONCLUSION

The Educational Organisation Management System successfully demonstrates how a low-code platform like ServiceNow can transform school administration. By implementing a relational table structure and automated business logic, the project has achieved its goal of creating a faster, more reliable, and paperless management environment.

10. FUTURE SCOPE

- **Mobile Integration:** Developing a mobile app view for parents to check real-time student progress.
- **AI Integration:** Using Predictive Analytics to identify students who may need extra academic support based on historical trends.
- **Module Expansion:** Adding Library, Fee Management, and Attendance tracking modules to the existing architecture.

11. APPENDIX

The complete source code for this project is available in the following GitHub repository:

GitHub Repository Link
https://github.com/Dharmateja15/Educational-Organisation-Using-ServiceNow

Project video link

https://drive.google.com/file/d/1dyI3F1UE9TU9hk1E8saIqP44fNinFm3s/view?usp=sharing